CORE JAVA TRAINING · PART 1 OF 25

Java Foundations

From zero to understanding exactly how a Java program runs

Difficulty	Easy
Prepared for	Mitansh Kanani
Covers	JDK/JRE/JVM · compilation & bytecode · class loading · JVM memory
	Hello World line by line · printing & printf · command-line args
	comments · keywords · identifiers · naming · reading errors
Assignments	8 coding tasks + 1 bonus
Tested on	JDK 21 (all code and error messages verified)

Contents

1	What is Java?
2	JDK, JRE and JVM
3	From .java to a running program
4	Setting up your machine
5	Hello World, line by line
6	Printing output
7	Command-line arguments
8	Comments, keywords, identifiers and naming
9	Reading errors like a professional
10	Self-check questions
11	Cheat sheet
12	Assignments

0 How to use this PDF

This is Part 1 of 25. It assumes you know **nothing** about Java. Even if some of this feels familiar, read every section. The goal is not just to know *what* things are, but to be able to explain *why* they work the way they do, and to write the code yourself without help.

Box	What it means
Trainer's tip	A habit or shortcut used by experienced developers.
Common mistake	Something almost every fresher gets wrong. Learn it here, not in your evaluation.
Under the hood	What Java is doing internally. This is what separates you from other freshers.
Your evaluator might ask	A question seniors like to ask, with the answer they expect.

How to study this part:

1. Read a section, then type every code example yourself. Do not copy-paste. Typing builds memory.
2. Compile and run everything from the **terminal** in this part, not with an IDE's Run button.
3. Break things on purpose. Remove a semicolon, misspell a word, and read the error. Errors are teachers.
4. Do the self-check questions in Section 10 without looking at the answers first.
5. Then solve the assignments in Section 12 and push them to GitHub.

> **Note**
> All code and every error message in this PDF were actually compiled and run on JDK 21. Your output should match exactly, except a few internal numbers that are noted where they appear.

1 What is Java?

1.1 Java in one paragraph

Java is a general-purpose, **object-oriented** programming language. You write code in **.java** files. A tool called the **compiler** turns that code into an intermediate format called **bytecode** (stored in **.class** files). Then a program called the **Java Virtual Machine (JVM)** reads that bytecode and runs it on your computer. Because every operating system has its own JVM, the same .class file can run on Windows, Linux or macOS without changes.

Java is used for backend servers, enterprise systems (banks, insurance, security products), Android apps, big-data tools (Hadoop, Kafka, Spark) and desktop tools. Many large companies have Java codebases that are 10 to 20 years old, which is why older technologies like Servlets and JSP are still worth learning.

1.2 A short history

Java was created by **James Gosling** and his team at **Sun Microsystems**. It was first called *Oak*, then renamed *Java* and publicly released in **1995**. **Oracle** bought Sun in 2010 and now leads Java's development. Since 2017, a new Java version comes out every six months. Some versions are marked **LTS (Long-Term Support)**, which means they get updates for many years. Companies almost always use LTS versions.

Version	Year	Why it matters
Java 8 (LTS)	2014	Lambdas and Streams. Still very common in older company code.
Java 11 (LTS)	2018	Run a single .java file directly. Oracle stopped shipping a separate JRE.
Java 17 (LTS)	2021	Records, sealed classes, text blocks, better switch.
Java 21 (LTS)	2023	Virtual threads, pattern matching for switch.
Java 25 (LTS)	2025	Simpler beginner programs (compact source files, instance main).

> **Trainer's tip**
> In your first week, ask your trainer which Java version the company's product uses. Features from newer versions may not compile on an older one, so write code for the version your team uses.

1.3 Key features of Java

Feature	What it actually means
Platform independent	Compiled bytecode runs on any OS that has a JVM. You do not recompile for each OS.
Object-oriented	Code is organised into classes and objects (Parts 7 to 11).
Statically typed	Every variable has a fixed type checked at compile time. Many bugs are caught before running.
Strongly typed	Java does not silently mix types in dangerous ways; you must cast explicitly.

Feature	What it actually means
Automatic memory management	You create objects; the **Garbage Collector** frees unused ones. No manual free() like C.
Robust	Strict compile checks, exception handling, no direct pointer arithmetic.
Secure	Bytecode is verified before it runs; no raw memory access.
Multithreaded	Built-in support to run many tasks at the same time (Part 19).
Huge standard library	Ready-made classes for strings, collections, files, networking, dates and more.

Your evaluator might ask

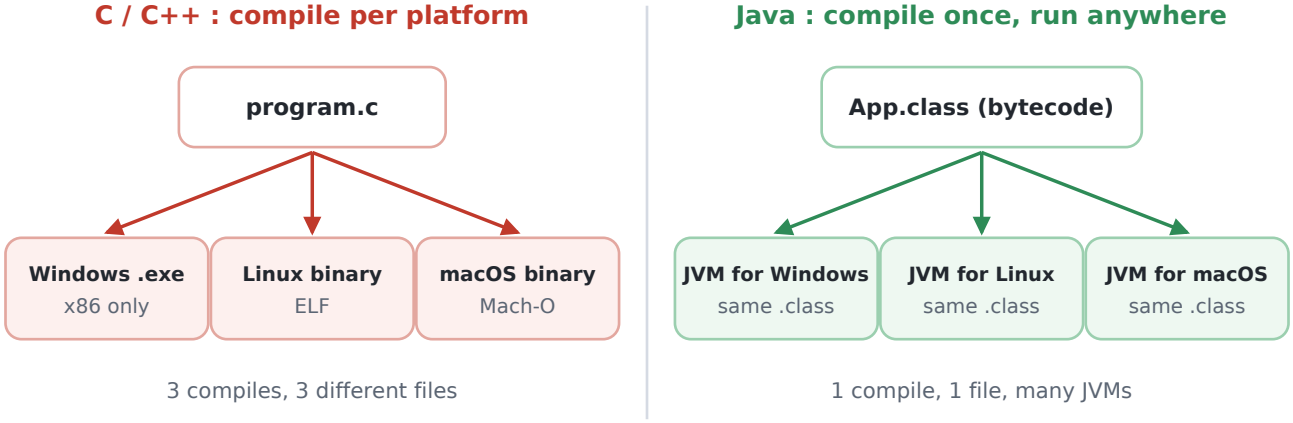
Q: Is Java 100% object-oriented?

No. Java has **primitive types** (int, char, boolean, etc.) that are not objects. It also has static members, which belong to a class rather than an object. That is why Java is called object-oriented but not *purely* object-oriented.

1.4 Platform independence: “Write Once, Run Anywhere”

To understand why Java is special, compare it with C. A C compiler turns your code directly into **machine code** for one specific operating system and processor. A Windows .exe will not run on Linux. You must compile again on each platform.

Java adds a middle layer. The compiler produces **bytecode**, which is not tied to any real processor. It is the instruction set of an imaginary computer: the Java Virtual Machine. Oracle and others write a JVM for every platform. So your one .class file runs anywhere a JVM exists.



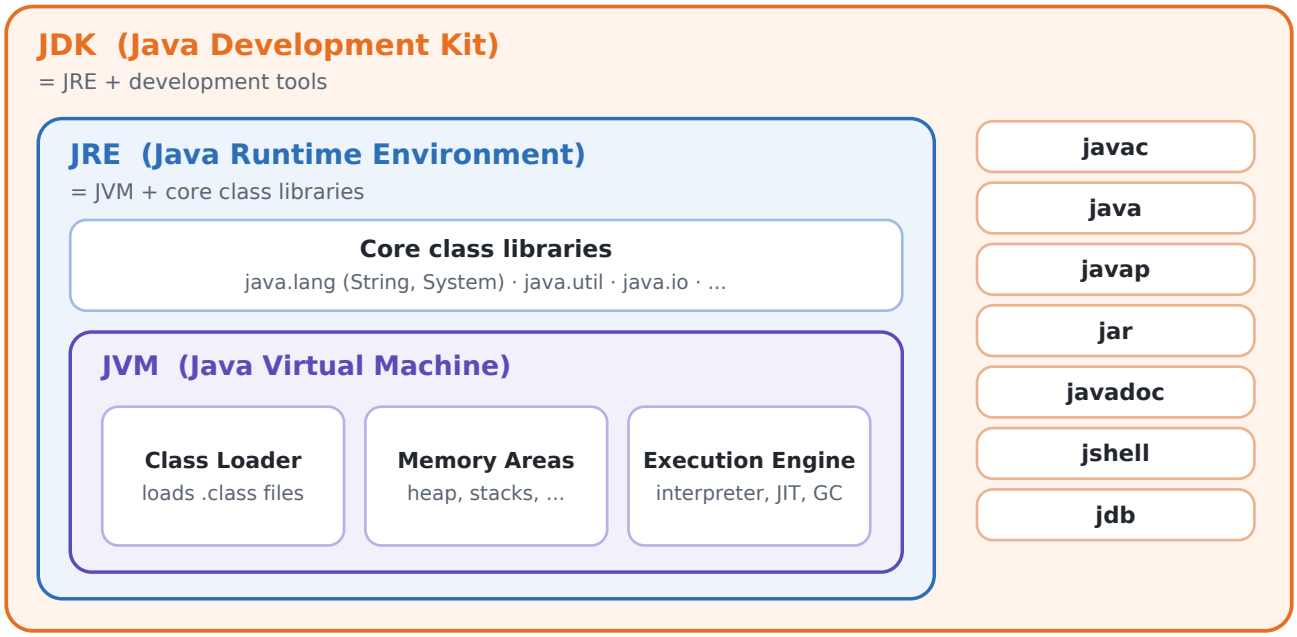
Your evaluator might ask

Q: Java is platform independent. Is the JVM platform independent too?

No. The JVM is **platform dependent**. There is a different JVM for Windows, Linux and macOS, each written for that system. The **bytecode** is what is platform independent. The JVM is the piece that makes that possible.

2 JDK, JRE and JVM

These three terms are asked about in almost every Java interview. Think of them as boxes inside boxes: the JVM is inside the JRE, and the JRE is inside the JDK.



2.1 JVM: Java Virtual Machine

The JVM is the program that actually **runs** your bytecode. It is called “virtual” because it behaves like a computer with its own instruction set, but it exists only as software. When you type `java HelloWorld`, the operating system starts a JVM process, and the JVM loads and runs your class.

The JVM has three main parts, which you will see in detail in Section 3:

- **Class Loader**: finds .class files and loads them into memory.
- **Runtime memory areas**: the heap (objects), stacks (method calls), and more.
- **Execution engine**: the interpreter, the JIT compiler, and the garbage collector.

> **Under the hood**
> The word “JVM” has two meanings. It can mean the **specification** (a document describing how any JVM must behave), or a real **implementation** of it. The most common implementation is **HotSpot**, which is inside OpenJDK and Oracle JDK. When you check your Java version in Section 4.3, “64-Bit Server VM” in the output is HotSpot.

2.2 JRE: Java Runtime Environment

The JVM alone is not enough to run a real program. Your code uses ready-made classes such as `String`, `System` and `ArrayList`. These live in the **Java class library**. The JRE is the JVM **plus** this library. In short, the JRE is what you need to **run** Java programs, but not to write them.

2.3 JDK: Java Development Kit

The JDK is the JRE **plus** development tools. As a developer, the JDK is what you install.

Tool	What it does	Example
`javac`	Compiler. Converts .java source into .class bytecode.	`javac Hello.java`
`java`	Launcher. Starts a JVM and runs a class.	`java Hello`
`javap`	Disassembler. Shows what is inside a .class file.	`javap -c Hello`
`jar`	Packs many .class files into one .jar archive.	`jar cf app.jar *.class`
`javadoc`	Generates HTML documentation from /** */ comments.	`javadoc Hello.java`
`jshell`	Interactive shell. Type Java and see results instantly (Java 9+).	`jshell`
`jdb`	Command-line debugger.	`jdb Hello`

Note

Before Java 11, you could download a JRE separately (for users who only run Java apps). Since Java 11, Oracle ships only the JDK. Tools like **jlink** can build a small custom runtime if needed. Some vendors still offer JRE builds, but for you, **always install the JDK**.

Your evaluator might ask

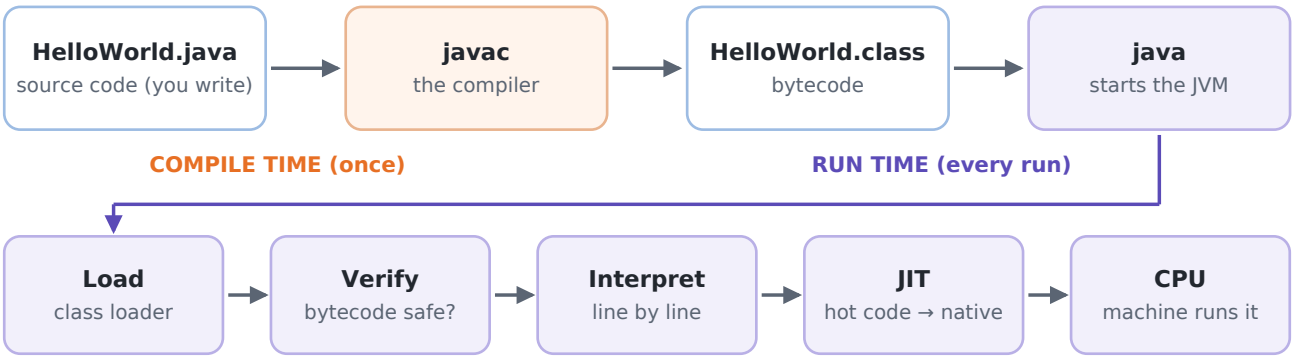
Q: Explain JDK, JRE and JVM in one line each.

JVM runs bytecode. **JRE** = JVM + class libraries, enough to run Java programs. **JDK** = JRE + tools like javac, needed to develop Java programs.

Q: Can you run a Java program with only the JDK installed? Yes. The JDK contains everything the JRE has.

3 From .java to a running program

Running a Java program always has two separate steps: **compile** (once, after each change) and **run** (as many times as you want). Understanding this flow is what your trainer means by knowing how Java works internally.



3.1 Step 1: Compile with javac

Here is the program we will follow through the whole journey:

HelloWorld.java

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Terminal

```
> javac HelloWorld.java
> dir                       (on Linux/macOS: ls)
HelloWorld.class   HelloWorld.java
```

When you run `javac HelloWorld.java`, the compiler does these jobs:

1. **Reads** the source text and splits it into words (keywords, names, symbols).
2. **Checks the syntax**: are braces closed, semicolons present, keywords spelled right?
3. **Checks the meaning**: does `System` exist? Does `println` accept a String? Are types used correctly?
4. If everything is correct, it **writes bytecode** into `HelloWorld.class`. If not, it prints errors and creates **no** .class file.

Common mistake

You compile a **file**, so you type the extension: `javac HelloWorld.java`. But you run a **class**, so you do not: `java HelloWorld`. Typing `java HelloWorld.class` gives: `Error: Could not find or load main class HelloWorld.class`.

3.2 What is bytecode? Let’s look inside

A .class file is binary, so you cannot read it in a text editor. The `javap -c` tool translates it into readable instructions:

Terminal

```
> javap -c HelloWorld
Compiled from "HelloWorld.java"
public class HelloWorld {
  public HelloWorld();
    Code:
       0: aload_0
       1: invokespecial #1      // Method java/lang/Object."<init>":()V
       4: return

  public static void main(java.lang.String[]);
    Code:
       0: getstatic     #7      // Field java/lang/System.out:Ljava/io/PrintStream;
       3: ldc           #13     // String Hello, World!
       5: invokevirtual #15     // Method java/io/PrintStream.println:(Ljava/lang/String;)V
       8: return
}
```

Read the `main` method line by line:

Instruction	Meaning in plain English
`getstatic #7`	Get the static field `System.out` (a PrintStream object) and put it on the operand stack.
`ldc #13`	Load the constant String “Hello, World!” from the class’s constant pool onto the stack.
`invokevirtual #15`	Call `println(String)` on that PrintStream, using the String as the argument.
`return`	The method is finished. Since main returns void, nothing is returned.

Under the hood

Two things to notice in this output:

1. A constructor you never wrote. `public HelloWorld();` appears even though your code has no constructor. If a class has no constructor, the compiler adds a **default constructor** that just calls the parent class’s constructor (`Object."<init>"`). Every class in Java extends `Object` if you don’t say otherwise. You will study this in Part 7 and Part 9.

2. The #numbers. `#7`, `#13` and `#15` are positions in the **constant pool**, a table inside every .class file storing names and constants. These numbers may differ on your machine depending on JDK version. That is normal.

Note

The JVM is a **stack-based** machine. Instructions push values onto an *operand stack* and pop them off to use them. Real CPUs mostly use registers. This is one reason bytecode is portable: it does not assume any real CPU’s registers.

3.3 Step 2: Run with java

When you type `java HelloWorld`, a lot happens before your first line runs:

(a) The JVM starts

The OS starts the JVM process. The JVM sets up its memory areas and loads core classes like `java.lang.Object` and `java.lang.String`.

(b) Class loading: Load → Link → Initialize

Phase	What happens
Loading	The class loader finds `HelloWorld.class` (in the current folder, by default) and reads its bytes into memory.
Linking: Verify	The **bytecode verifier** checks the bytecode is valid and safe. For example, it checks that no instruction jumps outside the method or misuses a type. This is a big part of Java’s security.
Linking: Prepare	Memory is allocated for static variables and set to default values (0, null, false).
Linking: Resolve	Symbolic names like `java/lang/System` are turned into real memory references (often lazily, when first used).
Initialization	Static variables get their real values and `static { }` blocks run. (Part 7.)

There are three built-in class loaders, arranged as parent and child:

Class loader	Loads	Example
Bootstrap	Core Java classes. Built into the JVM itself.	`java.lang.String`
Platform	Other Java SE modules. (Called *Extension* loader before Java 9.)	`java.sql.*`
Application (System)	Your classes, found on the **classpath**.	`HelloWorld`

> **Under the hood**
> Class loaders use **parent delegation**. When asked to load a class, a loader first asks its parent. Only if the parent cannot find it does the child try. That is why you cannot replace `java.lang.String` with your own fake version: the bootstrap loader always loads the real one first.

(c) Memory areas

You will use these ideas in every part. For now, know their names and purpose:

Area	Stores	Shared by threads?
Method Area (Metaspace)	Class information: method code, field names, constant pool, static variables.	Yes
Heap	All **objects** created with `new`, including arrays and Strings. Cleaned by the garbage collector.	Yes
Java Stack	One per thread. Each method call gets a **frame** holding its local variables and operand stack.	No
PC Register	One per thread. The address of the bytecode instruction currently running.	No
Native Method Stack	Used when Java calls native (C/C++) code.	No

(d) Execution engine

- **Interpreter**: reads bytecode one instruction at a time and executes it. Starts fast, but slow for code that runs many times.
- **JIT (Just-In-Time) compiler**: watches which methods run often (“hot” code). It compiles them into real **native machine code** and caches it. Later calls run at full CPU speed. This is why Java is both portable *and* fast.
- **Garbage Collector**: finds objects in the heap that nothing refers to anymore and frees their memory.

> **Your evaluator might ask**
> **Q: Is Java compiled or interpreted?**
> Both. Source code is **compiled** to bytecode by javac. The JVM then **interprets** bytecode and **JIT-compiles** hot parts into native code at runtime.

(e) Your main method runs

The JVM looks for the exact method `public static void main(String[] args)` in HelloWorld, creates a stack frame for it, and starts executing. When main finishes, and no

other non-daemon threads are running, the JVM shuts down.

3.4 Shortcut: running a single source file

Since Java 11, you can skip javac for a single-file program:

```
Terminal

> java HelloWorld.java
Hello, World!
```

This compiles the file **in memory** and runs it. No .class file is written. It is handy for quick experiments, but for this training always use the two-step `javac` then `java` flow, because real projects are built that way.

4 Setting up your machine

4.1 Install a JDK

Several companies build the same OpenJDK source. Any of these is fine for learning:

- **Eclipse Temurin** (from adoptium.net): free, widely used. Recommended.
- **Oracle JDK** (from oracle.com/java).
- **Amazon Corretto**, **Microsoft Build of OpenJDK**, and others.

Pick **JDK 21 (LTS)** unless your trainer tells you the company version. On Windows, choose the .msi installer and, if offered, tick **“Set JAVA_HOME variable”** and **“Add to PATH”**.

4.2 PATH and JAVA_HOME

When you type `javac` in a terminal, the OS searches the folders listed in the **PATH** environment variable for a program named javac. If the JDK’s `bin` folder is not in PATH, you get:

```
Windows Command Prompt

'javac' is not recognized as an internal or external command,
operable program or batch file.
```

To fix it manually on Windows:

1. Find your JDK folder, for example `C:\Program Files\Eclipse Adoptium\jdk-21.x.x-hotspot`.
2. Search Windows for **“Edit the system environment variables”** → **Environment Variables**.
3. Under System variables, click **New**: name `JAVA_HOME`, value = the JDK folder.
4. Edit **Path** → **New** → add `%JAVA_HOME%\bin`. Move it to the top if another Java exists.
5. **Close and reopen** the terminal. Old terminals keep the old PATH.

> **Note**
> `JAVA_HOME` is not needed by javac itself, but tools like Maven, Tomcat and IDEs use it to find your JDK. You will need it in Parts 22 to 24, so set it now.

4.3 Verify the installation

Terminal

```
> java -version
openjdk version "21.0.x" ...
OpenJDK Runtime Environment ...
OpenJDK 64-Bit Server VM ...

> javac -version
javac 21.0.x
```

Common mistake

If `java -version` works but `javac -version` does not, you probably have only a JRE on PATH, or an old Java from another program. Make sure the JDK's bin folder is first in PATH. Both commands should show the **same** version.

4.4 Your training workspace

Create one folder for the whole training. Each part gets its own subfolder:

Folder structure

```
java-training/
├── part-01/
│   ├── HelloWorld.java
│   └── ...
├── part-02/
└── .gitignore
```

Terminal

```
> cd java-training\part-01
> javac HelloWorld.java
> java HelloWorld
Hello, World!
```

You must be **inside the folder** that contains the .class file when you run `java`, because by default the classpath is the current folder (`.`).

4.5 Editor and jshell

For Part 1, use any plain editor (VS Code is fine) but compile and run from the terminal. Later you can install the **Extension Pack for Java** in VS Code. Learning the terminal first means you will understand what the IDE's Run button does behind the scenes.

For quick experiments, use **jshell**. It runs single lines without needing a class or main:

Terminal

```
> jshell
jshell> 7 / 2
$1 ==> 3
jshell> "Java" + 10 + 20
$2 ==> "Java1020"
jshell> /exit
```

> **Trainer's tip**
> Use jshell to test small doubts ("what does this print?") in seconds. Use .java files for anything you want to keep or submit.

5 Hello World, line by line

HelloWorld.java

```
public class HelloWorld {                       // line 1
    public static void main(String[] args) {  // line 2
        System.out.println("Hello, World!");  // line 3
    }                                          // line 4
}                                              // line 5
```

5.1 Line 1: public class HelloWorld {

Word	Meaning
`public`	An **access modifier**. This class can be used from anywhere. (Full details in Part 8.)
`class`	A keyword that says "I am defining a class." In Java, all code must live inside a class.
`HelloWorld`	The class name, chosen by you. By convention it uses PascalCase.
`{ ... }`	Curly braces mark the start and end of the class body. Everything inside belongs to the class.

A **class** is a blueprint. Later you will create objects from it. For now, think of it as the container that holds your code.

Rules about files and classes

- If a class is `public`, the file name **must** exactly match the class name, including upper and lower case: `HelloWorld` → `HelloWorld.java`.
- A file can contain **at most one** public class.
- A file can contain many **non-public** classes. Each class becomes its **own .class file** after compiling.
- A file with no public class can have any name, but by convention you still name it after the main class.

Public class Hello saved in Wrong.java

```
Wrong.java:1: error: class Hello is public, should be declared in a file named Hello.java
public class Hello {
       ^
1 error
```

5.2 Line 2: public static void main(String[] args)

This is the **entry point**. When you run `java HelloWorld`, the JVM looks for this exact method and starts executing from its first line. Every word has a reason:

Part	Why it is needed
`public`	The JVM is outside your class. It must be allowed to call main, so main must be public.
`static`	The JVM calls main **before any object exists**. A static method belongs to the class itself, so it can be called without creating an object. Without static, the JVM would need to create a HelloWorld object first, and it would not know which constructor to use.
`void`	main returns nothing to the JVM. To send an exit status to the OS, use `System.exit(0)` (success) or a non-zero number (failure).
`main`	The exact name the JVM searches for. `Main` or `mian` will not work. Java is case-sensitive.
`String[] args`	An array of Strings holding the **command-line arguments** (Section 7). The name `args` is only a convention; any valid name works.

Valid and invalid ways to write main

Signature	Result
`public static void main(String[] args)`	**Valid.** The standard form. Always write this.
`public static void main(String args[])`	Valid. C-style array syntax. Works but not recommended.
`public static void main(String... args)`	Valid. Varargs form (Part 5).
`static public void main(String[] args)`	Valid. Modifier order does not matter. Unusual style.
`public static void main(final String[] args)`	Valid. Parameters can be final.
`public static void main(String[] x)`	Valid. The parameter name can be anything.
`public void main(String[] args)`	Compiles, but on JDK 21: `Error: Main method is not static in class G`.
`public static int main(String[] args)`	Compiles, but: `Error: Main method must return a value of type void`.
`public static void main(string[] args)`	Compile error: `cannot find symbol ... class string`.
`public static void Main(String[] args)`	Compiles, but: `Error: Main method not found in class`.

> **Note**
> Java 25 introduced **instance main methods** and **compact source files** for beginners. There, `void main()` without public, static or args can work. Company code and interviews use the classic form, so always write `public static void main(String[] args)`.

Your evaluator might ask

Q: Why is main static? So the JVM can call it without creating an object of the class.

Q: Can a program compile without main? Yes. javac does not need main. The error appears only at **runtime**, when `java` cannot find an entry point.

Q: Can we overload main? Yes, you can write `main(int x)` too. But the JVM only calls the `String[]` version as the entry point.

Q: Can two classes in one project both have main? Yes. You choose which one to start: `java ClassA` or `java ClassB`.

5.3 Line 3: System.out.println("Hello, World!");



Piece	What it is
`System`	A class in the `java.lang` package. java.lang is imported automatically into every file, which is why you never write an import for it.
`out`	A **static field** inside System. Its type is `PrintStream`. It is connected to the standard output: your terminal.
`println`	A method of PrintStream. Prints its argument, then moves to a new line.
`"Hello, World!"`	A **String literal**. Text inside double quotes.
`;`	Ends the statement. Every statement in Java ends with a semicolon.

Under the hood

Because `out` is static, you access it through the class name: `System.out`, without creating a System object. This is the same reason main is static. You will meet this pattern again and again in Java.

Common mistake

Java is **case-sensitive**. All of these fail:

`system.out.println` → `error: package system does not exist`

`System.out.printn` → `error: cannot find symbol symbol: method printn(String)`

`System.Out.println` → `error: cannot find symbol`

5.4 Lines 4 and 5: closing braces

The first `}` closes main. The second closes the class. Every `{` needs a matching `}`. Indenting each level by 4 spaces makes the structure easy to see. The compiler does not care about indentation, but your evaluators will.

5.5 Many classes in one file

Zoo.java

```
public class Zoo {
    public static void main(String[] args) {
        System.out.println("Welcome to the Zoo");
    }
}

class Lion {
    public static void main(String[] args) {
        System.out.println("Roar");
    }
}
```

Terminal

```
> javac Zoo.java
> dir
Lion.class   Zoo.class   Zoo.java
> java Zoo
Welcome to the Zoo
> java Lion
Roar
```

One .java file produced **two** .class files. The JVM works with classes, not files, so you can start either class as long as it has a main method.

6 Printing output

6.1 print, println and printf

PrintDemo.java

```
public class PrintDemo {
    public static void main(String[] args) {
        System.out.print("A");          // no new line after
        System.out.print("B");
        System.out.println("C");        // new line after
        System.out.println();           // just a blank line
        System.out.println("D");
        System.out.printf("Pi is about %.2f%n", 3.14159);
    }
}
```

Output

```
ABC

D
Pi is about 3.14
```

Method	Behaviour
`print(x)`	Prints x. The cursor stays on the same line.
`println(x)`	Prints x, then a line separator. `println()` with no argument prints just a new line.

Method	Behaviour
`printf(format, values...)`	Prints using a **format string** with placeholders like %d and %s. Does **not** add a new line; use %n.

6.2 Escape sequences

Some characters cannot be typed directly inside a String. Use a backslash to “escape” them:

Escape	Meaning	Example code	Prints
`\n`	New line	`"Line1\nLine2"`	Line1 Line2
`\t`	Tab	`"Name\tAge"`	Name Age
`\"`	Double quote	`"He said \"Hi\""`	He said "Hi"
`\'`	Single quote	`'\'' (a char)`	'
`\\`	Backslash	`"C:\\Users"`	C:\Users
`\r`	Carriage return	`(rarely used directly)`	—
`\u0041`	Unicode character	`"\u0041"`	A

Trainer’s tip

In printf, prefer `%n` over `\n`. `%n` uses the correct line separator for the operating system (Windows uses \r\n, Linux and macOS use \n).

6.3 Joining text with +

When either side of `+` is a String, Java **concatenates** (joins) them as text. When both sides are numbers, it adds them. Java evaluates `+` from **left to right**, and this causes surprising results:

Concatenation puzzle (verified output in comments)

```
System.out.println(1 + 2 + "3" + 4 + 5);   // 3345
System.out.println("Java" + 10 + 20);       // Java1020
System.out.println("Java" + (10 + 20));     // Java30
System.out.println(10 + 20 + "Java");       // 30Java
```

Take the first line step by step:

1. `1 + 2` → both numbers → `3`.
2. `3 + "3"` → one side is a String → `"33"`.
3. `"33" + 4` → String + number → `"334"`.
4. `"334" + 5` → `"3345"`.

Common mistake

Once a String appears, everything after it becomes text. If you want a sum inside a message, use parentheses: `"Total: " + (a + b)`.

6.4 printf format specifiers

Specifier	Type	Example	Output
`%d`	Integer	`printf("%d", 42)`	42
`%f`	Decimal (6 places by default)	`printf("%f", 3.14)`	3.140000
`%.2f`	Decimal, 2 places (rounded)	`printf("%.2f", 3.14159)`	3.14
`%s`	String (or any object)	`printf("%s", "Java")`	Java
`%c`	Character	`printf("%c", 'J')`	J
`%b`	Boolean	`printf("%b", true)`	true
`%e`	Scientific notation	`printf("%e", 12345.678)`	1.234568e+04
`%%`	A literal % sign	`printf("100%%")`	100%
`%n`	Platform new line	`printf("Hi%n")`	Hi + new line

Width and alignment

A number between % and the letter sets the **minimum width**. A minus sign means **left-align**. This is how you print neat tables:

```
System.out.printf("|%10s|%-10s|%5d|%8.2f|%n", "Java", "Code", 42, 3.14159);
```

Output

```
|      Java|Code      |   42|    3.14|
```

Part	Meaning
`%10s`	"Java" right-aligned in 10 characters (6 spaces + 4 letters).
`%-10s`	"Code" left-aligned in 10 characters.
`%5d`	42 right-aligned in 5 characters.
`%8.2f`	3.14159 rounded to 2 decimals, right-aligned in 8 characters.

Common mistake

If the value type does not match the specifier, you get a **runtime** error, not a compile error. For example, `printf("%d", 3.5)` throws `IllegalFormatConversionException`. The compiler does not check format strings.

Note

`String.format("%.2f", 3.14159)` works exactly like printf, but **returns** the text as a String instead of printing it. Useful when you need the text later.

Note

`System.err.println(...)` prints to the **standard error** stream. It looks the same in a terminal but is a separate stream, used for error messages. Many IDEs show it in red.

7 Command-line arguments

Anything you type after the class name in the `java` command is passed into `main` as the `args` array. Words are separated by spaces.

Greet.java

```
public class Greet {
    public static void main(String[] args) {
        System.out.println("Number of arguments: " + args.length);
        System.out.println("First: " + args[0]);
        System.out.println("Second: " + args[1]);
    }
}
```

Terminal

```
> java Greet Mitansh Mumbai
Number of arguments: 2
First: Mitansh
Second: Mumbai
```

Command	args contents	args.length
`java Greet`	`{ } (empty array, not null)`	0
`java Greet Mitansh`	`{ "Mitansh" }`	1
`java Greet Mitansh Mumbai`	`{ "Mitansh", "Mumbai" }`	2
`java Greet "Mitansh Kanani" Mumbai`	`{ "Mitansh Kanani", "Mumbai" }`	2
`java Greet 10 20`	`{ "10", "20" } (Strings, not numbers!)`	2

Notice: arguments are counted from **index 0**, and quotes join words that contain spaces.

7.1 Arguments are always Strings

Even if you type numbers, `args` holds text. To do maths, convert them with `Integer.parseInt` (for whole numbers) or `Double.parseDouble` (for decimals):

Doubler.java

```
public class Doubler {
    public static void main(String[] args) {
        int a = Integer.parseInt(args[0]);   // "21" -> 21
        System.out.println(a * 2);
    }
}
```

Terminal

```
> java Doubler 21
42
```

Here `int a` creates a **variable** that holds a whole number. Variables and types are covered fully in Part 2. For now, it is enough to know that `int` holds whole numbers and `double` holds

decimals.

7.2 What can go wrong

Terminal

```
> java Doubler
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
    Index 0 out of bounds for length 0
at Doubler.main(Doubler.java:3)

> java Doubler ten
Exception in thread "main" java.lang.NumberFormatException: For input string: "ten"
at java.base/java.lang.NumberFormatException.forInputString(...)
at java.base/java.lang.Integer.parseInt(Integer.java:662)
...
at Doubler.main(Doubler.java:3)
```

- **ArrayIndexOutOfBoundsException**: you asked for `args[0]` but no argument was passed. Arrays (Part 4) only have the positions they were created with.
- **NumberFormatException**: `"ten"` is not a valid number, so parseInt cannot convert it.

Trainer's tip

How to read a stack trace: read it from the **top line** (what went wrong) and then find the **first line that mentions your own file**. Here, `at Doubler.main(Doubler.java:3)` tells you the problem is on line 3 of your file. The lines starting with `java.base/` are inside Java's own code.

You will learn to prevent these crashes with if-checks (Part 3) and exception handling (Part 12).

Note

In VS Code, arguments are set in `.vscode/launch.json` using an `"args"` entry. In this part, pass them from the terminal.

8 Comments, keywords, identifiers and naming

8.1 Comments

Three kinds of comments

```
// Single-line comment: everything after // on this line is ignored.

/* Multi-line comment.
   Can span many lines.
   Cannot be nested: /* inside another */ breaks it. */

/**
 * Javadoc comment. Used to generate HTML documentation.
 * @param args command-line arguments
 */
public static void main(String[] args) { }
```

The compiler removes all comments; they never appear in bytecode. Write comments that explain **why**, not *what*. `// add 1 to i` is useless. `// skip the header row` is helpful.

Generating Javadoc

```
> javadoc -d docs HelloWorld.java
> (open docs/index.html in a browser)
```

8.2 Keywords

Keywords are words reserved by Java. You cannot use them as names. All keywords are lowercase.

`abstract`	`assert`	`boolean`	`break`	`byte`	`case`
`catch`	`char`	`class`	`const`	`continue`	`default`
`do`	`double`	`else`	`enum`	`extends`	`final`
`finally`	`float`	`for`	`goto`	`if`	`implements`
`import`	`instanceof`	`int`	`interface`	`long`	`native`
`new`	`package`	`private`	`protected`	`public`	`return`
`short`	`static`	`strictfp`	`super`	`switch`	`synchronized`
`this`	`throw`	`throws`	`transient`	`try`	`void`
`volatile`	`while`	`_`			

- `const` and `goto` are reserved but **not used**. Java keeps them so nobody uses them as names.
- `true`, `false` and `null` look like keywords but are technically **literals**. You still cannot use them as names.
- **Contextual keywords** like `var`, `record`, `yield`, `sealed` and `permits` are special only in certain places. Avoid them as names anyway.

8.3 Identifiers

An **identifier** is any name you create: class, method, variable or package names. The rules (broken rules = compile error):

- May contain letters, digits, `_` (underscore) and `$` (dollar).
- Must **not start with a digit**.
- Must not be a keyword or literal (`true`, `false`, `null`).
- No spaces or symbols like `-`, `@`, `#`.
- Case-sensitive: `total`, `Total` and `TOTAL` are three different names.
- No length limit, but keep names readable.

Identifier	Valid?	Reason
`studentName`	Yes	Letters only.
`_count`	Yes	May start with underscore (but avoid by convention).
`$price`	Yes	Allowed, but $ is meant for generated code. Avoid.
`total2`	Yes	Digits allowed after the first character.

Identifier	Valid?	Reason
`2total`	No	Starts with a digit.
`my-name`	No	Hyphen is the minus operator.
`class`	No	Keyword.
`first name`	No	Contains a space.
`_`	No	Since Java 9, a single underscore is a keyword.

8.4 Naming conventions

Conventions are not enforced by the compiler, but every professional Java team follows them. Breaking them in code review makes you look like a beginner.

Element	Convention	Examples
Class / Interface	PascalCase, nouns	Student, BankAccount, FileReader
Method	camelCase, verbs	calculateTotal(), getName(), isValid()
Variable	camelCase, meaningful	firstName, totalMarks, count
Boolean variable	is / has / can prefix	isActive, hasLicense, canEdit
Constant	UPPER_SNAKE_CASE	MAX_SIZE, PI, DEFAULT_TIMEOUT
Package	all lowercase, reversed domain	com.company.project.util

> **Trainer's tip**
> Avoid single-letter names except for short loop counters (`i`, `j`). `int d;` tells the reader nothing. `int daysUntilExpiry;` tells them everything.

9 Reading errors like a professional

9.1 Compile-time vs runtime errors

	Compile-time error	Runtime error
Found by	javac, before the program runs	the JVM, while the program runs
.class created?	No	Yes (the code compiled fine)
Examples	Missing semicolon, wrong spelling, type mismatch	Missing argument, bad number format, no main method
How it looks	`File.java:3: error: ...`	`Exception in thread "main" ...` or `Error: ...`

> **Trainer's tip**
> Compile-time errors are your friends. Java's strict checking catches mistakes **before** a customer ever sees them. This is a big reason companies like Java.

9.2 Anatomy of a compiler error

Terminal

```
A.java:3: error: ';' expected
        System.out.println("Hi")
                                ^
1 error
```

- `A.java:3`: the file and **line number**.
- `error: ';' expected`: what the compiler wanted.
- The code line, then ^: the caret points to **where** the compiler noticed the problem.
- `1 error`: total count.

Common mistake

Two habits that save hours:

1. Fix the first error first, then recompile. One missing brace can cause ten fake errors below it.

2. The error line is where Java noticed the problem, not always where you made it. A missing `}` is often reported at the very end of the file.

9.3 Common beginner errors (real messages)

You did this	You see this	Fix
javac not on PATH	`'javac' is not recognized as an internal or external command`	Add JDK bin to PATH (Section 4.2). Reopen terminal.
Public class name ≠ file name	`class Hello is public, should be declared in a file named Hello.java`	Rename the file or the class so they match exactly.
Forgot a semicolon	`error: ';' expected`	Add ; at the caret position.
Lowercase system	`error: package system does not exist`	Write System with a capital S.
Misspelled method	`error: cannot find symbol symbol: method printn(String)`	Check spelling: println.
Lowercase string	`error: cannot find symbol symbol: class string`	Write String with a capital S.
Forgot closing quote	`error: unclosed string literal`	Close the String with ".
Missing closing brace	`error: reached end of file while parsing`	Match every { with a }.
Ran java X.class	`Error: Could not find or load main class X.class`	Run java X (no extension).
Wrong folder	`Error: Could not find or load main class X`	cd into the folder with X.class.
No main method	`Error: Main method not found in class F, please define the main method as: public static void main(String[] args)`	Add a correct main method.

You did this	You see this	Fix
main not static	`Error: Main method is not static in class G`	Add static.
No argument given	`ArrayIndexOutOfBoundsException: Index 0 out of bounds for length 0`	Pass the argument, or check args.length first.
Non-number to parseInt	`NumberFormatException: For input string: "ten"`	Pass a valid number.

10 Self-check questions

Answer each question out loud or on paper **before** reading the answer. If you get one wrong, re-read the section in brackets.

#	Question	Answer
1	What does javac produce, and what does the JVM consume?	javac produces .class files containing bytecode. The JVM loads and runs that bytecode. (3.1)
2	Why can the same .class file run on Windows and Linux?	Bytecode is not tied to any real CPU or OS. Each platform has its own JVM that understands the same bytecode. (1.4)
3	Difference between JDK, JRE and JVM?	JVM runs bytecode. JRE = JVM + class libraries. JDK = JRE + development tools like javac. (2)
4	Name the three phases of class loading.	Loading, Linking (verify, prepare, resolve), Initialization. (3.3)
5	What is the JIT compiler and why does it exist?	It compiles frequently-run bytecode into native machine code at runtime, so hot code runs at full CPU speed. (3.3)
6	Where do objects live, and where do local variables live?	Objects live on the heap. Local variables live in the method’s frame on the thread’s stack. (3.3)
7	Why must main be public and static?	Public so the JVM can access it from outside the class. Static so the JVM can call it without creating an object. (5.2)
8	Does a program without main compile?	Yes. The error only appears at runtime when java looks for the entry point. (5.2)
9	You put two non-public classes and one public class in one file. How many .class files do you get?	Three. Every class gets its own .class file. (5.5)
10	What does System.out.println really consist of?	System is a class, out is its static PrintStream field, println is a method of PrintStream. (5.3)
11	What does System.out.println(5 + 5 + "5" + 5) print?	"1055". 5+5=10 first, then everything becomes text. (6.3)
12	What is args.length when no arguments are passed? Is args null?	0. args is an empty array, never null, when launched with java. (7)
13	Is 2ndPlace a valid identifier? Is _ ?	No: it starts with a digit. No: _ is a keyword since Java 9. (8.3)

#	Question	Answer
14	Compile-time vs runtime error: which one leaves a .class file?	A runtime error. The code compiled fine; it failed while running. (9.1)
15	What is in the constant pool?	Names of classes, methods and fields, and constants like String literals, referenced by #numbers in bytecode. (3.2)

11 Cheat sheet

Part 1 at a glance

```
COMPILE          javac FileName.java          -> FileName.class (bytecode)
RUN              java ClassName               (no .class extension!)
RUN ONE FILE     java FileName.java           (Java 11+, compiles in memory)
SEE BYTECODE     javap -c ClassName
TRY CODE         jshell
DOCS             javadoc -d docs FileName.java

ENTRY POINT      public static void main(String[] args) { ... }

PRINT            System.out.print(x)          no new line
                 System.out.println(x)        new line after
                 System.out.printf("%d %s %.2f%n", 1, "a", 2.5)

ESCAPES          \n  \t  \"  \'  \\  \uXXXX
FORMAT           %d int  %f decimal  %.2f  %s text  %c char  %b boolean
                 %10s right-align  %-10s left-align  %n new line  %% percent

ARGS             args.length   args[0]   Integer.parseInt(args[0])

JDK = JRE + tools (javac, javap, jar, javadoc, jshell)
JRE = JVM + class libraries
JVM = class loader + memory areas + execution engine (interpreter, JIT, GC)

NAMES            ClassName   methodName   variableName   CONSTANT_NAME   com.pkg.name
```

12 Assignments

Solve these **without AI and without copy-pasting from the internet**. Looking at this PDF is allowed. Compile and run every program from the terminal.

Submission rules

- Create a GitHub repository named `java-training` (public, so it can be reviewed).
- Put all Part 1 files directly inside a `part-01` folder. One public class per file, named exactly as given.
- Add a `.gitignore` in the repo root containing `*.class`. Never commit compiled files.
- Add `part-01/README.md` with the exact commands you used to compile and run each program, and its output. This proves you ran it.
- Commit after each assignment with a clear message, for example `"Part 1 - A4 Greeter"`.

- Code quality counts: 4-space indentation, meaningful names, a short comment at the top of each file saying what it does.

A1. ProfileCard.java (Easy)

Print a profile card exactly like this, using only `println`. Use your own details.

Expected output

```
+----------------------------------+
|  NAME   : Mitansh Kanani         |
|  ROLE   : Java Trainee           |
|  CITY   : Mumbai                 |
|  GOAL   : Master Core Java       |
+----------------------------------+
```

Then: rewrite the same card using **one single** `println` call and `\n` escape sequences. Keep both versions in the file and print a blank line between them.

A2. EscapeMaster.java (Easy)

Print the following output **exactly**. Every quote, backslash and tab must appear as shown. Columns in the last two lines are separated by tabs (`\t`).

Expected output

```
He said, "Java is fun!"
It's Mitansh's first program.
File path: C:\Users\Mitansh\java-training\part-01
Name	Role	City
Mitansh	Trainee	Mumbai
```

Bonus: print the letter `J` using its Unicode escape (`\u004A`).

A3. PredictTheOutput.java (Easy)

Copy these lines into main. **Before running**, write your predicted output as a comment at the end of each line. Then run it. For any line you got wrong, add a second comment explaining **why** the real output is what it is.

Lines to predict

```
System.out.println(10 + 20 + "Java");
System.out.println("Java" + 10 + 20);
System.out.println("Java" + (10 + 20));
System.out.println(1 + 2 + "3" + 4 * 5);
System.out.println("" + 7 + 3);
System.out.println(7 + 3 + "");
System.out.println(7 / 2);
System.out.println(7.0 / 2);
System.out.println(7 % 2);
System.out.print("A");
System.out.println("B");
System.out.print("C\n");
System.out.println("D\tE");
```

Trainer's tip

Line 4 uses *. Multiplication happens before +, just like in maths. Think about the order carefully.

A4. Greeter.java (Easy)

Take a name and a city as command-line arguments and print a greeting.

Terminal

```
> java Greeter Mitansh Mumbai
Hello, Mitansh from Mumbai!
You passed 2 arguments.

> java Greeter "Mitansh Kanani" "Navi Mumbai"
Hello, Mitansh Kanani from Navi Mumbai!
You passed 2 arguments.
```

In README.md, also note what happens when you run `java Greeter` with no arguments, and copy the exact error message.

A5. ArgsCalculator.java (Easy–Medium)

Take two whole numbers as arguments. Convert them with `Integer.parseInt`. Print all five results using `printf` (not println).

Terminal

```
> java ArgsCalculator 17 5
17 + 5 = 22
17 - 5 = 12
17 * 5 = 85
17 / 5 = 3
17 % 5 = 2
```

Then: add a sixth line that prints the **exact** decimal division with 2 decimal places, using `Double.parseDouble` instead: `17 / 5 = 3.40`. In a comment, explain why the integer version printed 3.

A6. Receipt.java (Medium)

Print a shop receipt using `printf` with widths and alignment. Item names left-aligned, numbers right-aligned, prices with 2 decimals. Values can be written directly in the code (no input needed).

Expected output (alignment must match)

```
============================================
ITEM                QTY      PRICE      TOTAL
============================================
Notebook              3      45.50     136.50
Pen                  10      10.00     100.00
Java Book             1     599.99     599.99
--------------------------------------------
GRAND TOTAL                             836.49
============================================
```

Hint: work out each column's width by counting characters. You may calculate totals by hand and type them in for now; in Part 2 you will let Java calculate them.

A7. ErrorLab (ERRORS.md) (Medium)

Start from a working `HelloWorld.java`. Make **one** change at a time from the list below, compile and run, then undo it. Record your findings in `part-01/ERRORS.md`.

1. Remove the semicolon after println.
2. Change `System` to `system`.
3. Remove the word `static` from main.
4. Rename `main` to `Main`.
5. Rename the file to `Hello.java` but keep the class name HelloWorld.
6. Remove the last closing brace `}`.
7. Remove the closing double quote of the String.
8. Change `void` to `int` (and add `return 0;`).
9. Run `java HelloWorld.class` instead of `java HelloWorld`.
10. Remove `public` from the class (keep it on main). Does it still run?

For each case, write a table row with: the change, **compile-time or runtime?**, the **exact error message** (copy it), and one sentence on **why** it happens.

A8. Zoo.java + Bytecode notes (Medium)

Create `Zoo.java` containing one public class `Zoo` and three non-public classes `Lion`, `Parrot` and `Snake`. Each class has its own main that prints a different message. Compile once, then run all four classes.

Then run `javap -c Lion` and answer in `README.md`:

1. How many .class files did one javac command create? Why?
2. Which bytecode instruction loads your String message? Which one calls println?
3. The javap output shows a constructor `Lion();` that you never wrote. Where did it come from, and what does it call?
4. Why can you run `java Lion` even though Lion is not public?

A★. Bonus: Documented.java (Optional)

Write a class with a Javadoc comment on the class and on main (use `@param` and `@author`). Generate HTML docs with `javadoc -d docs Documented.java`, open `docs/index.html` in a browser and take a screenshot. Add the screenshot to the repo. Do **not** commit the generated docs folder; add `docs/` to .gitignore.

> **What happens next**
>
> When you are done, share your repository link. Each assignment will be reviewed for correctness, output matching exactly, code style and naming, and how well your explanations show that you understand the *why*.